

PersonalHub

A personal Task, Routine, File & Expense Manager — built with Django

Purpose of this document	Full architecture & code reference
Audience	Any AI assistant or developer continuing this project
Backend framework	Django 5.0.6 (Python)
Database	MySQL (Aiven, cloud-hosted)
Hosting	Render.com
File storage	Cloudinary
Email delivery	Brevo Transactional API
Scheduler	cron-job.org (free, external)

Contents

1. What This Project Is	Overview
2. System Architecture	How the pieces connect
3. Data Model	Every table & relationship
4. User Accounts & Approval Flow	Login, register, admin approval
5. Feature: Tasks & Treasure Box	Daily to-dos
6. Feature: Daily Routine	Recurring checklist
7. Feature: Transactions	Income/expense tracker
8. Feature: Files & Notes	Uploads + text notes
9. Reminder & Email Pipeline	Cron → Brevo
10. Superuser: Users Page	Approval + storage usage
11. Deployment & Infrastructure	Render, Aiven, Cloudinary, Brevo
12. Environment Variables	Full reference table
13. Project File Map	What every file does
14. Known Issues & Inconsistencies	Things to fix next
15. Running & Deploying	Step-by-step commands

1. What This Project Is

PersonalHub is a personal productivity web app built in Django. One person (or a small approved group) logs in and manages four things in one place:

 **Tasks**

One-off to-dos with an optional email reminder. Completed tasks move into a "Treasure Box" instead of disappearing.

 **Transactions**

Income/expense tracking with user-defined categories, filtering, and a quick $+-\times\div$ calculator on the amount field.

 **Daily Routine**

Recurring checklist items that reset every day, with daily progress percentages and an optional daily reminder time.

 **Files & Notes**

Upload files (stored on Cloudinary so they survive redeployments) and create quick text notes with copy buttons.

On top of these, there's a lightweight **account approval system** (new sign-ups wait for a superuser to approve them) and a **superuser dashboard** showing all users plus an estimate of how much of the database's 1 GB quota each one is using.

WHY THIS DOCUMENT EXISTS

This project was built across several AI-assisted sessions, each one adding features on top of the last. This document is the single source of truth for the *current, final state* of the code — so that you, or any other AI assistant, can pick it up cold and know exactly how it works without re-reading the entire chat history.

2. System Architecture

PersonalHub is a classic Django monolith, but it leans on **four external cloud services** because the app itself runs on Render's free tier, which has two important restrictions: its disk is wiped on every redeploy, and it blocks outbound SMTP (port 587). Every external service below exists specifically to work around one of those two limits.

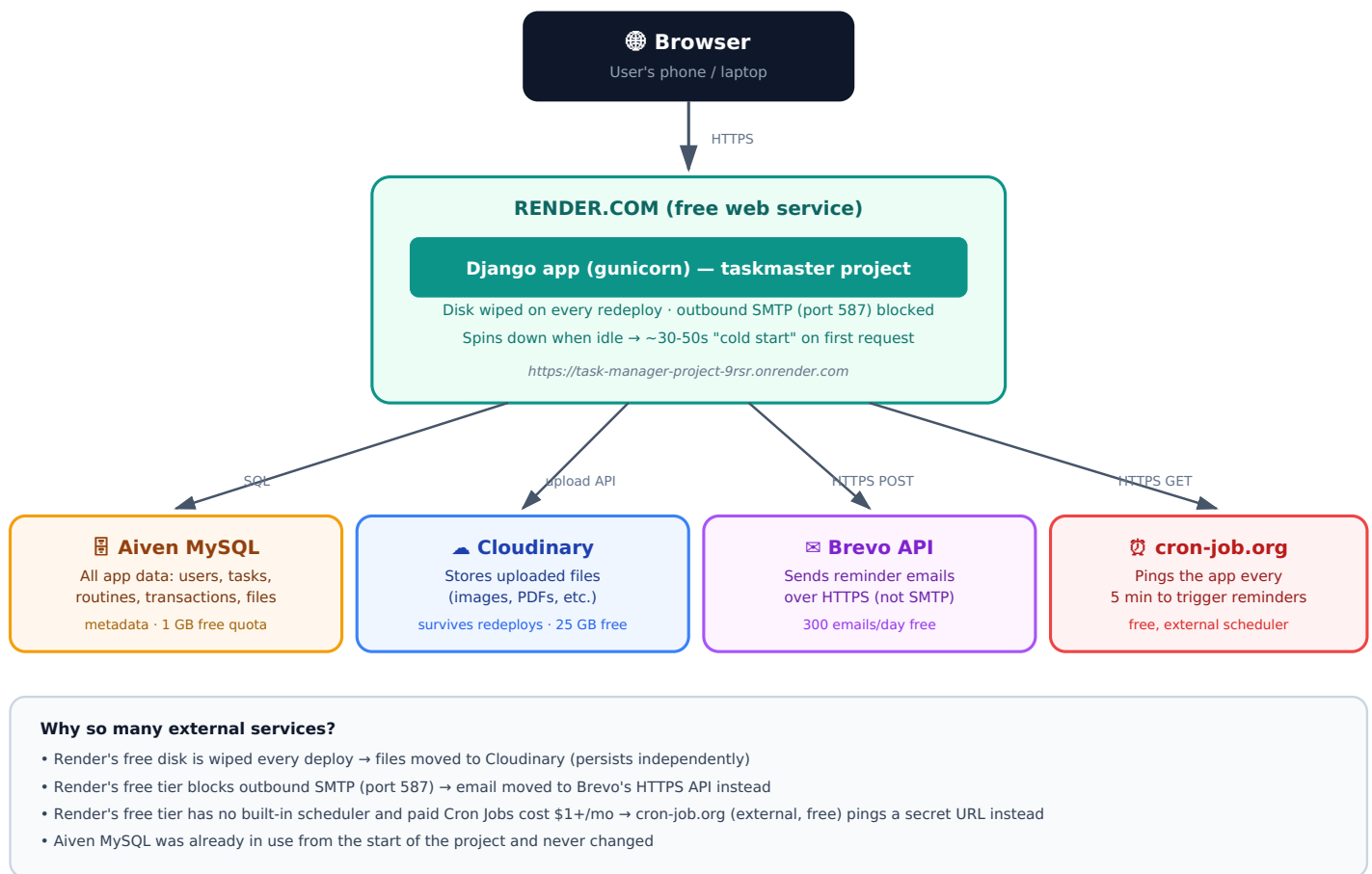


Fig. 1 — High-level architecture: one Django app on Render, talking to four external cloud services

Request lifecycle (a normal page load)

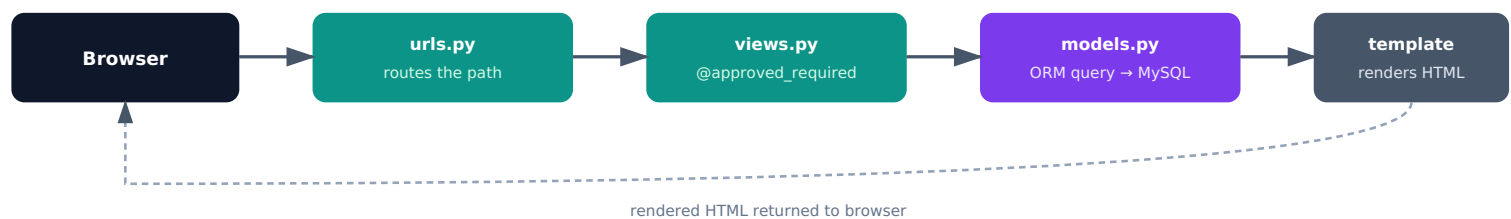


Fig. 2 — Standard Django request/response cycle used by every page in this app

3. Data Model

Every table in the database is defined as a Django model in `tasks/models.py`. There are **8 models**, all ultimately owned by Django's built-in `User` table. The diagram below shows every table and how they connect.

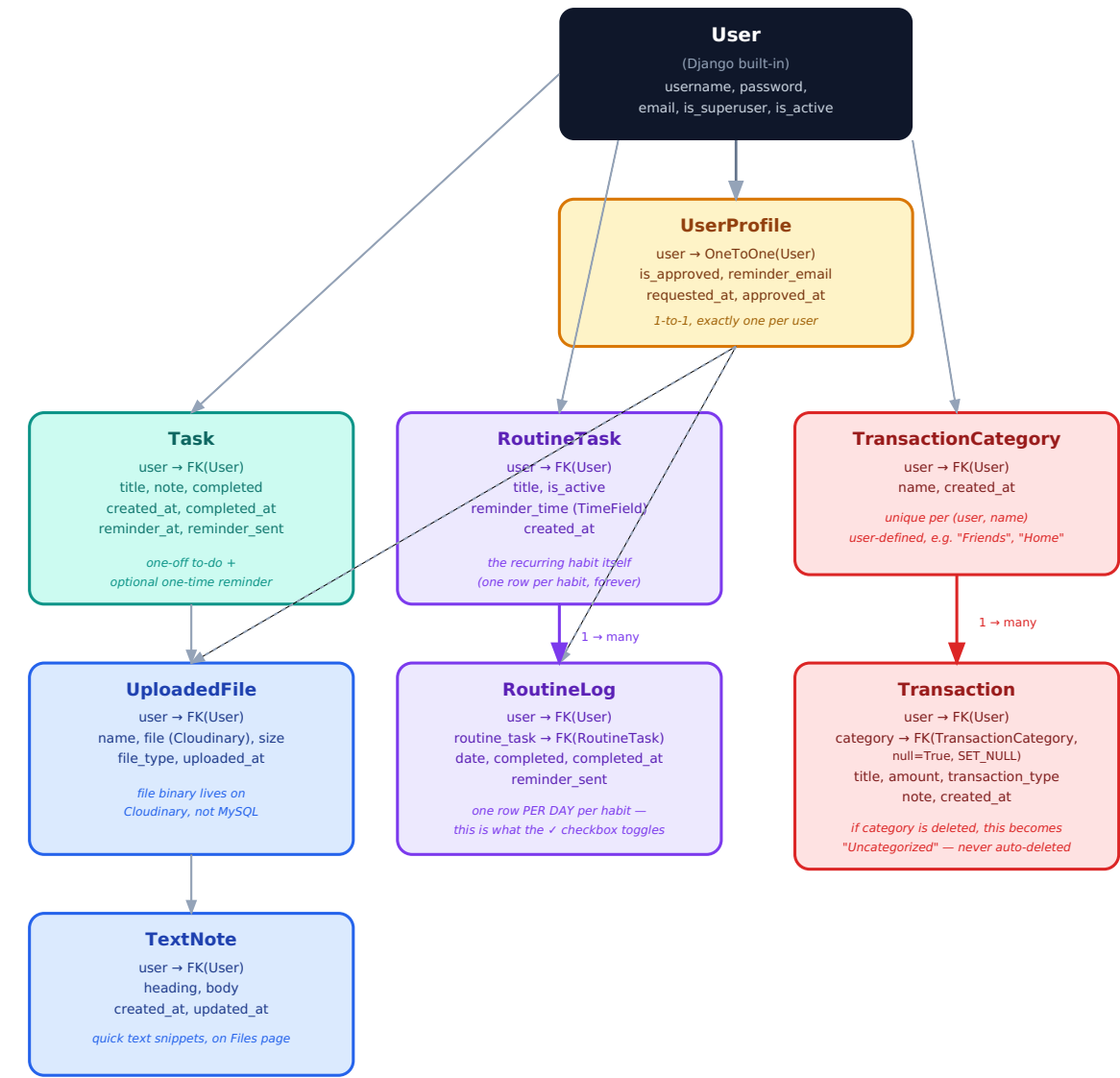


Fig. 3 — Entity-relationship diagram of all 8 models. Solid grey arrows = "belongs to a User". Dashed = via `UserProfile` lookup only (no DB FK).

The trickiest relationship: RoutineTask vs. RoutineLog

This is the one relationship worth slowing down on, because it's not obvious from the name alone.

MENTAL MODEL

RoutineTask = the habit definition itself ("Morning workout") — created once, lives forever.
RoutineLog = one row *per day* that habit is checked. Every midnight (technically: every time the Routine page loads on a new day), a fresh `RoutineLog` row is created for each active `RoutineTask`, with `completed=False`. Ticking the checkbox just flips that one day's row to `True` — it never touches yesterday's row. This is how "today resets but history is kept" works without any special reset logic.

Model reference table

Model	Key fields	Purpose
<code>UserProfile</code>	<code>is_approved, reminder_email</code>	Extends Django's User with approval status + where to send reminder emails
<code>Task</code>	<code>title, completed, reminder_at</code>	One-off to-do items, optionally with a one-time reminder
<code>RoutineTask</code>	<code>title, reminder_time</code>	A recurring daily habit definition
<code>RoutineLog</code>	<code>date, completed (unique per task+date)</code>	One checkbox state per habit per calendar day
<code>UploadedFile</code>	<code>name, file, size, file_type</code>	Metadata for a file whose actual bytes live on Cloudinary

TextNote	heading, body	Quick text snippets created directly in-app (no upload)
TransactionCategory	name (unique per user)	User-defined labels like "Friends", "Home", "Food"
Transaction	amount, transaction_type, category	A single income or expense entry

4. User Accounts & Approval Flow

Anyone can register an account, but they can't actually use the app until a superuser approves them. This was a deliberate design choice — it's a personal app, not a public sign-up page, so it needed a gate.

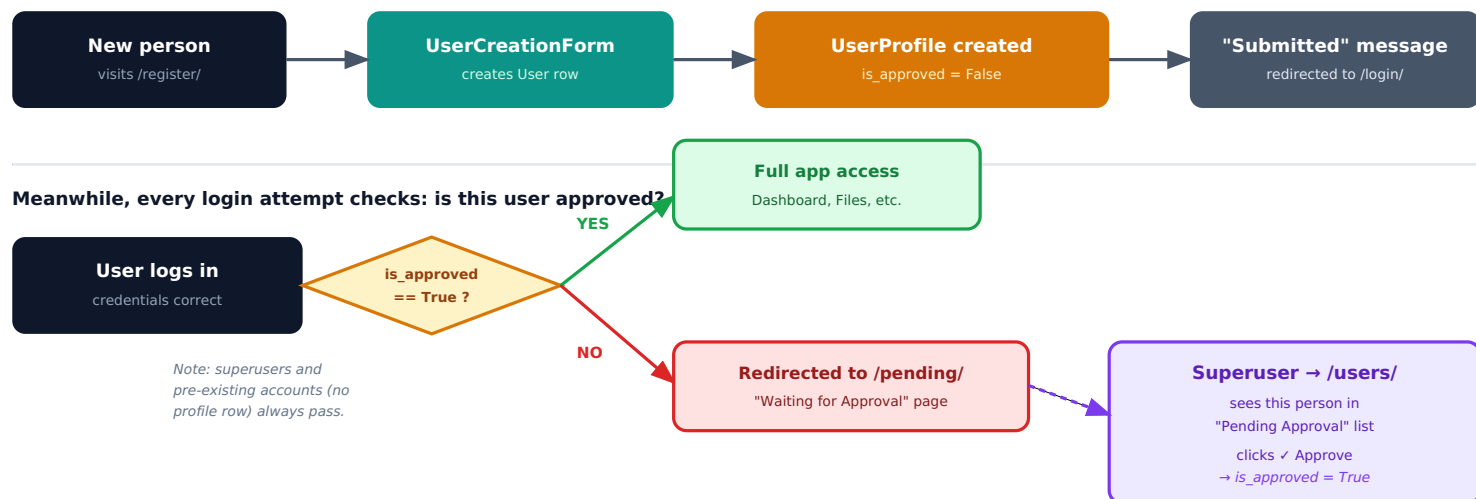


Fig. 4 — Registration → approval gate → superuser review loop

The gatekeeper: `approved_required`

This is a custom decorator in `views.py` that wraps `@login_required`. Almost every view in the app uses it instead of plain `@login_required`:

```
def approved_required(view_func):
    """Like @login_required, but also blocks users pending admin
    approval, sending them to the waiting-room page instead."""
    @login_required
    def wrapper(request, *args, **kwargs):
        if not _is_approved(request.user):
            return redirect('pending_approval')
        return view_func(request, *args, **kwargs)
    return wrapper
```

EDGE CASE HANDLED ON PURPOSE

Superusers and any account that existed *before* this feature was built (and therefore has no `UserProfile` row at all) are both treated as automatically approved. This stops the original admin account from accidentally locking itself out the day this feature shipped.

5. Feature: Tasks & Treasure Box

The dashboard (/) shows active tasks. Marking one complete doesn't delete it — it moves into a collapsible "Treasure Box" section, which is really just the same table filtered by `completed=True` .

Action	URL	What happens
Add task	<code>POST /</code>	Creates a <code>Task</code> row; optional <code>reminder_at</code> datetime
Complete	<code>POST /task/complete/<pk>/</code>	Sets <code>completed=True</code> , stamps <code>completed_at</code>
Edit	<code>POST /task/edit/<pk>/</code>	Inline form on the same page (JS confirm → reveal form)
Restore	<code>POST /task/restore/<pk>/</code>	Moves a task back out of the Treasure Box
Delete one / all	<code>POST /task/delete/...</code>	Permanently removes task row(s)

KNOWN BUG — NAIVE DATETIME

The reminder date/time field uses an HTML `<input type="datetime-local">` , which submits a plain string like `2026-06-25T13:52` with **no timezone attached**. Django's settings have `USE_TZ = True` and `TIME_ZONE = 'Asia/Kolkata'` , so every datetime is expected to carry timezone info. Because this one doesn't, Django logs `RuntimeWarning: DateTimeField Task.reminder_at received a naive datetime` and the reminder may fire at a shifted time. See Section 14 for the recommended fix.

6. Feature: Daily Routine

This is the "recurring habit" feature, and its mechanics were already explained in Section 3 — `RoutineTask` is the habit, `RoutineLog` is one day's checkbox. The view logic that makes "today always exists" work is a small helper called on every page load:

```
def _ensure_today_logs(user):
    today = date.today()
    active_routines = RoutineTask.objects.filter(user=user, is_active=True)
    for rt in active_routines:
        RoutineLog.objects.get_or_create(
            routine_task=rt, date=today, defaults={'user': user}
        )
```

`get_or_create` means this is safe to call every single time the Routine page loads — if today's row already exists, nothing happens; if it doesn't (i.e. this is the first visit since midnight), it's created fresh with `completed=False`. **There is no scheduled "reset" job** — the reset is really just "yesterday's row stays, today's row is new."

The page also computes 3 progress percentages (today / this week / this month) by counting `RoutineLog` rows in each date range — pure aggregate queries, no extra model needed.

7. Feature: Transactions

Income/expense tracking with two notable design decisions: user-defined categories (not a fixed list), and a small calculator attached to the amount field.

Categories are per-user and freeform

`TransactionCategory` has `unique_together = ('user', 'name')` — so two different users can both have a category called "Food" without clashing, but the same user can't create "Food" twice. When a category is deleted, its transactions are **not** deleted — `on_delete=models.SET_NULL` just clears the link, and the transaction shows up under "Uncategorized".

The +−×÷ amount calculator

This is pure client-side JavaScript — no server round-trip. Clicking $\times$, $\div$, $+$ or $-$ opens a tiny popover; typing a number and hitting "Apply" runs the arithmetic directly on the amount input's value:

```
function applyCalc(amountFieldId, operandFieldId, opFieldId, popoverId) {
  var current = parseFloat(amountEl.value) || 0;
  var operand = parseFloat(operandInput.value);
  if (op === 'mul') result = current * operand;
  else if (op === 'div') result = current / operand;
  else if (op === 'add') result = current + operand;
  else if (op === 'sub') result = current - operand;
  amountEl.value = Math.round(result * 100) / 100;
}
```

Example: type "100", set amount to "₹2", tap $\times$ → type "100" in the popover → Apply → amount becomes ₹200. (A server-side `/transactions/calc/` endpoint also exists doing the same math in Python, but the UI doesn't currently call it — it's a leftover/unused alternative path, safe to delete or repurpose.)

8. Feature: Files & Notes

This feature had the most painful bug in the whole project, so it's worth explaining *why* it's built the way it is, not just what it does.

THE ORIGINAL BUG: FILES "NOT OPENING"

Early on, uploaded files worked for a while, then started returning **404 Not Found**. The cause: Render's free-tier disk is **ephemeral** — every time the app redeploys (any `git push`), the entire filesystem resets to whatever's in the Git repo. Uploaded files lived at `/media/...` on that disk, so they were silently wiped on the very next deploy, even though the database still had a row pointing at the now-missing file.

The fix: file *bytes* now live on Cloudinary (a separate, persistent service), and MySQL only stores a reference to them. This is configured with one conditional line in `settings.py`:

```
if CLOUDINARY_STORAGE['CLOUD_NAME']:
    DEFAULT_FILE_STORAGE = 'cloudinary_storage.storage.MediaCloudinaryStorage'
```

When the 3 Cloudinary env vars are missing (e.g. on a fresh local dev setup), Django quietly falls back to local disk storage — so the app still runs locally without needing a Cloudinary account, it just won't have persistent storage in that mode.

FILES UPLOADED BEFORE THIS FIX ARE UNRECOVERABLE

Any file uploaded while the bug was active is gone — its database row still exists, but `download_file` will raise a friendly 404 explaining it needs to be re-uploaded. This is expected, not a new bug.

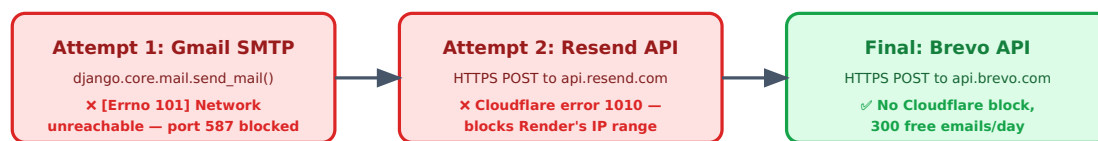
Text Notes — a smaller, simpler feature on the same page

`TextNote` is unrelated to file storage entirely — heading + body text, stored directly as MySQL columns (no Cloudinary involved), with a "Copy heading" / "Copy text" button using the browser's `navigator.clipboard` API.

9. Reminder & Email Pipeline

This is the most heavily reworked part of the whole project. It went through **three different email providers** before landing on the current design — each one was blocked by a Render free-tier restriction the previous attempt didn't anticipate. Understanding that history explains every odd-looking piece of the final code.

The journey: why Brevo, and why HTTPS instead of SMTP



The pattern across all 3 attempts

Render's free tier restricts outbound network access in ways that aren't documented up front. SMTP (port 587) is blocked outright. Some HTTPS APIs sit behind Cloudflare, which flags Render's shared IPs as bot traffic. The fix was trial-and-error until a provider worked — Brevo did.

Fig. 5 — Three email providers were tried in sequence; only the third avoided Render's network restrictions

Full pipeline: from "5 minutes pass" to "email arrives"

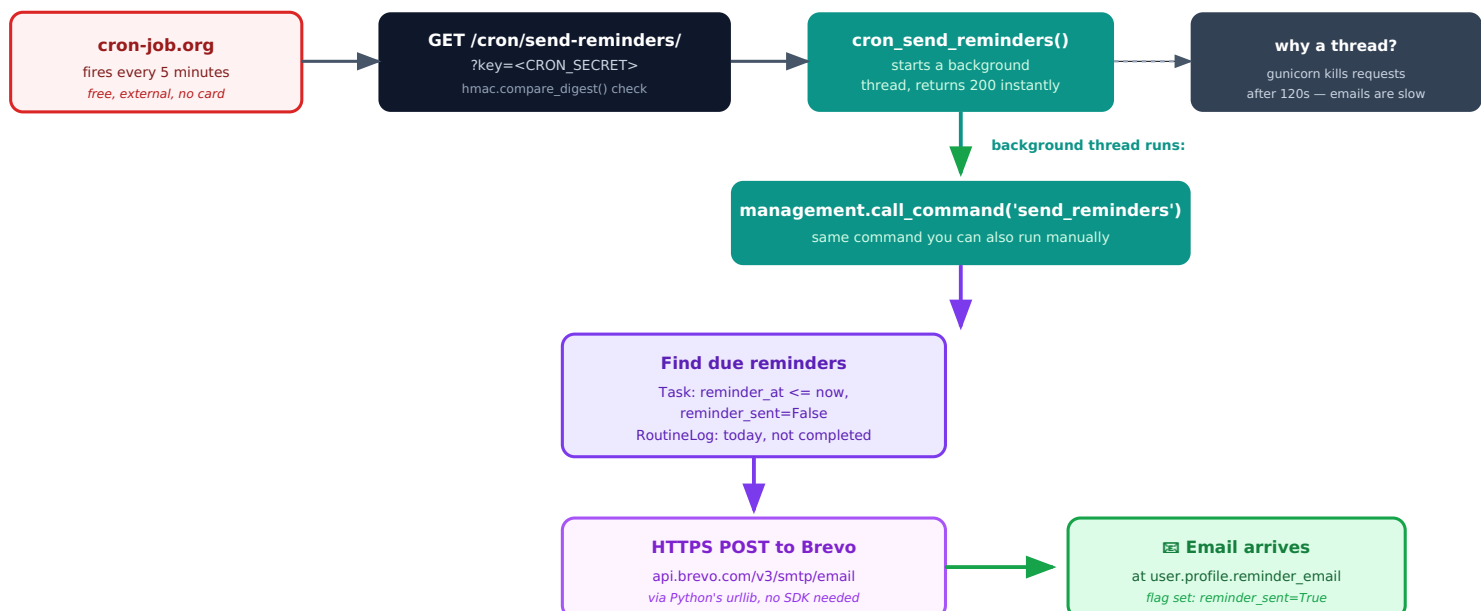


Fig. 6 — Full reminder pipeline, end to end. Every box maps to a real file in the repo (see Section 13).

Per-user, not per-app, email addresses

One Brevo account sends *all* reminder emails, but each user receives at their own address. The "from" address is hardcoded in `send_reminders.py`, while the "to" address is looked up per-user from `UserProfile.reminder_email` (set on the `/profile/` page):

```
def _email_for(self, user):
    profile = getattr(user, 'profile', None)
    if profile and profile.reminder_email:
        return profile.reminder_email
    return '' # no email on file → silently skipped
```

BREVO FREE-TIER SENDING RESTRICTION ENCOUNTERED DURING TESTING

On Brevo's free plan, the *sender* address must be a verified address in the Brevo account — it's currently hardcoded as `mahanthileelakrishnaprasad@gmail.com` inside `send_reminders.py` (not an env var). If this needs to change, it must also be re-verified inside the Brevo dashboard under Transactional → Senders & IP.

Why the `reminder_sent` flag exists on every reminder-capable model

The cron job runs every 5 minutes — without a "already sent" flag, the same reminder would re-fire every 5 minutes forever.

`Task.reminder_sent` and `RoutineLog.reminder_sent` both default to `False` and flip to `True` the moment an email is sent (or skipped because no address is on file). Editing a task's reminder time resets the flag back to `False` so it can fire again.

10. Superuser: Users Page

Visible only to `request.user.is_superuser`, this page (`/users/`) does two unrelated jobs: approving new sign-ups (Section 4), and estimating how much of the 1 GB Aiven MySQL quota each user's data occupies.

The storage estimate is approximate, by design

Getting an *exact* per-user byte count would require querying MySQL's internal storage metadata per row — slow and overkill for a personal app. Instead, `_estimate_user_bytes()` sums the character length of each user's text fields plus a small fixed overhead per row, and adds the real `size` field already stored for uploaded files:

```
for t in Task.objects.filter(user=user).only('title', 'note'):
    total += len(t.title or '') + len(t.note or '') + 64 # + row overhead

for f in UploadedFile.objects.filter(user=user).only('name', 'size'):
    total += (f.size or 0) + len(f.name or '') + 64
```

This repeats for every model, then the page shows it as both a raw size (`KB` / `MB`) and a percentage of the 1 GB quota, with a small horizontal usage bar per user plus one combined bar at the top of the page for the whole app.

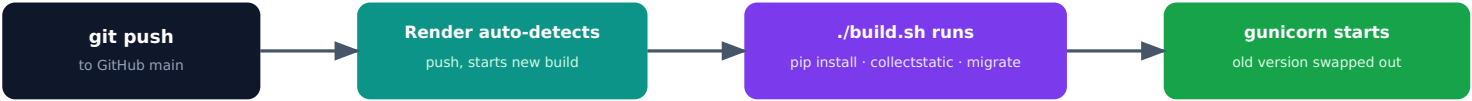
WHY THIS CAN NEVER BE PERFECTLY ACCURATE

It deliberately ignores MySQL index overhead, InnoDB page padding, and the fact that uploaded file *bytes* now live on Cloudinary, not MySQL — so the "storage usage" shown here is actually approximating *two different storage systems* at once (MySQL row data + Cloudinary file sizes) as if they were one pool. Good enough to spot a runaway user; not a billing-grade number.

11. Deployment & Infrastructure

Service	Role	Free tier limit	Sign up
Render	Hosts the Django app (web service)	Spins down when idle; 30-50s cold start; outbound SMTP blocked	render.com
Aiven	Managed MySQL database	1 GB storage	aiven.io
Cloudinary	Stores uploaded file bytes	25 GB storage, persists forever	cloudinary.com
Brevo	Sends reminder emails (HTTPS API)	300 emails/day	brevo.com
cron-job.org	Pings the reminder webhook every 5 min	Free, unlimited basic jobs	cron-job.org

The deploy pipeline (what happens on `git push`)



⚠ **Disk reset on every deploy**
Anything written to local disk (old-style file uploads, SQLite, etc.) is wiped here. This is exactly why Cloudinary exists — it's untouched by this step.

Fig. 7 — What actually happens between pushing code and the live site updating

Known inconsistency: `render.yaml` is stale

THIS FILE DOCUMENTS AN ABANDONED PLAN, NOT THE LIVE SETUP

`render.yaml` in the repo root still describes a **paid Render Cron Job** and **Gmail SMTP env vars** — both abandoned in favor of the free cron-job.org webhook and Brevo API. It's not actually wired up as a Render "Blueprint" either (the service was created by hand in the dashboard), so Render never reads this file at all currently. It's safe to ignore, delete, or rewrite to match reality — see Section 14.

12. Environment Variables

All set under Render → your web service → **Environment**. None of these are committed to Git.

Variable	Used for	Example / where to get it
SECRET_KEY	Django session/CSRF signing	any long random string
DEBUG	Django debug mode	False in production
DB_HOST / DB_PORT / DB_NAME / DB_USER / DB_PASSWORD	Aiven MySQL connection	Aiven console → service → Connection info
CLOUDINARY_CLOUD_NAME	File storage	Cloudinary dashboard home page
CLOUDINARY_API_KEY	File storage	Cloudinary dashboard home page
CLOUDINARY_API_SECRET	File storage	Cloudinary dashboard home page
BREVO_API_KEY	Sending reminder emails	Brevo → Settings → SMTP & API → API Keys
CRON_SECRET	Protects the /cron/send-reminders/ webhook	any random string, e.g. openssl rand -hex 32

UNUSED BUT STILL DEFINED: EMAIL_HOST_USER, EMAIL_HOST_PASSWORD

These were for the abandoned Gmail SMTP attempt (Section 9). settings.py still configures Django's built-in EMAIL_BACKEND with them, but no code path actually calls django.core.mail.send_mail() anymore — send_reminders.py talks to Brevo directly over HTTPS instead. Safe to remove both the env vars and the dead settings block.

13. Project File Map

```
Task_manager_project/
├─ manage.py — Django's CLI entry point
├─ build.sh — runs on every Render deploy
├─ Procfile — gunicorn start command (--timeout 120)
├─ requirements.txt — Python dependencies
├─ render.yaml — ▲ stale, not actually used (Sec. 11)
├─ runtime.txt — pins Python 3.11.0
├─
├─ taskmaster/ — Django project config
│ └─ settings.py — DB, Cloudinary, email, all env var wiring
│ └─ urls.py — root URL config, includes tasks.urls
│ └─ wsgi.py
├─
└─ tasks/ — the one Django "app" containing everything
    ├─ models.py — all 8 models (Section 3)
    ├─ views.py — every view function, ~680 lines
    ├─ urls.py — app-level routes
    ├─ admin.py — registers models with /admin/
    ├─ migrations/ — 4 migration files, see table below
    ├─ management/commands/
    │ └─ send_reminders.py — Brevo email sender (Section 9)
    ├─ templatetags/
    │ └─ tasks_extras.py — get_item filter, for Users page dict lookup
    └─ templates/tasks/ — one .html file per page, extend base.html
```

Migration history

File	What it added
0001_initial.py	Original Task, UploadedFile, RoutineTask, RoutineLog, Transaction
0002_transactioncategory_transaction_category.py	TransactionCategory + the category FK on Transaction
0003_userprofile.py	UserProfile (approval gate)
0004_reminders_and_textnotes.py	reminder_at/reminder_time/reminder_sent fields + TextNote model

views.py — function index

Function	Guarded by	Does
login_view / register_view	—	Auth entry points
dashboard	@approved_required	Task list + add form
complete_task / edit_task / delete_task / restore_task	@approved_required	Task CRUD
users_view	@superuser_required	Approve/reject users, storage usage
files_view / delete_file / download_file	@approved_required	File & TextNote CRUD
routine_view	@approved_required	Daily checklist + progress %
transactions_view / calc_amount	@approved_required	Income/expense CRUD + categories
profile_view	@approved_required	Set reminder email
cron_send_reminders	custom secret-key check	Webhook entry point for cron-job.org

14. Known Issues & Inconsistencies

Things found while writing this document that are real, but not urgent — useful to know about before touching the relevant code.

#	Issue	Where	Suggested fix
1	Naive datetime on task reminders triggers a runtime warning and may fire at the wrong time	<code>views.py</code> → <code>dashboard()</code> , <code>edit_task()</code>	Convert with <code>django.utils.timezone.make_aware()</code> before saving <code>reminder_at</code>
2	<code>render.yaml</code> documents a Gmail SMTP + paid Cron Job setup that was abandoned	<code>render.yaml</code>	Rewrite to describe <code>BREVO_API_KEY</code> + <code>CRON_SECRET</code> , or delete the file entirely since it isn't read
3	Dead Gmail SMTP config block left in settings (never executed, but misleading to read)	<code>settings.py</code> lines ~110-120	Delete <code>EMAIL_BACKEND</code> / <code>EMAIL_HOST*</code> block + the two env vars on Render
4	Brevo sender email is hardcoded in source rather than an env var	<code>send_reminders.py</code> line ~25	Move to <code>BREVO_SENDER_EMAIL</code> env var for portability
5	Unused server-side <code>/transactions/calc/</code> endpoint duplicates client-side JS calculator	<code>views.py</code> → <code>calc_amount()</code>	Either wire the UI to use it, or delete it — currently dead code
6	Render free tier cold-starts (30-50s) can make the first cron-job.org ping after idle time look like a failure	infra, not code	Accepted tradeoff of the free tier; a second "keep-alive" cron ping to <code>/</code> every 5 min reduces this
7	Brevo free tier requires the <i>sender</i> to be a verified address — changing it requires re-verifying in the Brevo dashboard	infra, not code	Documented here so it isn't a surprise next time

15. Running & Deploying

Run locally (Windows)

```
cd Task_manager_project
venv\Scripts\activate
pip install -r requirements.txt

# point at the same Aiven cloud DB used in production —
# there is no local MySQL in this setup
set DB_HOST=your-aiven-host.aivencloud.com
set DB_PORT=26149
set DB_NAME=defaultdb
set DB_USER=avnadmin
set DB_PASSWORD=your-aiven-password

python manage.py makemigrations
python manage.py migrate
python manage.py runserver
```

Open `http://127.0.0.1:8000` . Because the database is shared with production, any data you create locally is visible on the live site too — use obviously-fake test data.

Test the reminder pipeline locally

```
# Option 1 — run the command directly
python manage.py send_reminders

# Option 2 — test the actual HTTP webhook
set CRON_SECRET=testkey123
# then visit in browser:
# http://127.0.0.1:8000/cron/send-reminders/?key=testkey123
```

Also requires `BREVO_API_KEY` set, and a task with `reminder_at` in the past, to actually see an email sent rather than `Done. 0 reminder(s) sent.`

Deploy a change

```
git add .
git commit -m "describe the change"
git push
```

Render auto-deploys on every push to `main` . Watch progress at Render Dashboard → your service → **Logs**. `build.sh` runs migrations automatically, so any new migration file just needs to be committed — no manual `migrate` step on the server.

Set up the reminder cron (one-time, already done — for reference)

1. Sign up free at cron-job.org
2. New cronjob → URL: `https://<your-render-url>/cron/send-reminders/?key=<CRON_SECRET>`
3. Schedule: every 5 minutes · Timeout: 30 seconds (cron-job.org's max)
4. Save & enable — check the execution log for `200 OK` responses